FRONT-END DEVELOPMENT
HACKATHON – Level-2 Template (Interactivity Implementation)

STUDENT NAME: NIRANCHEZHIYAN R

YEAR: II BSC COMPUTERSCIENCE

COLLEGE CODE: 364

COLLEGE NAME: PURATCHITHALAIVAR DR.MGR GOVT. ARTS&SCIENCE.COLLEGE

DEPARTMENT: COMPUTER SCIENCE

SEMESTER: IV

ROLL NO: 36424U18027

NMID: astvu36436424u18027

DATE: 10.04.2026

TITLE: EXPENSE TRACKER

GITHUB LINK:

1. Objective

Describe the goal of implementing front-end interactivity to update activity status and display real-time progress using JavaScript or React.

2. Problem Focus

Explain the transition from Level-1 static UI to Level-2 interactive UI. Highlight the importance of event handling, state changes, and dynamic updates.

3. Application Overview

Provide a brief description of the application such as type of application (Task Tracker/Activity Tracker) and key features implemented.

4. Data Handling Approach
Explain how data is managed using static JSON, JavaScript array, React state, or optional Local Storage.

5. UI Components Design
Describe components such as Activity List, Buttons/Checkboxes, and Progress Display section.

6. Interactivity Implementation
Explain event handling, updating activity status from Pending to Completed, and dynamic UI updates.

7. Application Logic
Describe logic for counting completed activities, updating progress, and instant UI refresh without reload.

8. Technology Used
Mention HTML, CSS, JavaScript or React.

9. Best Practices Followed
Use meaningful names, avoid hardcoding, add comments, and handle edge cases.

10. Key Features Implemented
Mark activity as completed, real-time progress tracking, and dynamic UI updates.

11. Expected Output / Deliverables
Working front-end application, interactive tracking system, real-time progress display, and GitHub source code with proper structure.

12. Learning Outcome
JavaScript event handling, DOM manipulation or React basics, state management, and building interactive UI.

13. Conclusion
Summarize how the application successfully implements interactivity and improves user experience.

1. Objective

The main objective of the Expense Tracker application is to help users efficiently manage their daily expenses, track spending habits, and maintain financial discipline. It aims to provide a simple and intuitive interface for recording, categorizing, and analyzing expenses.

2. Problem Focus

Many individuals struggle with tracking where their money goes, leading to poor budgeting and overspending. Traditional methods like manual записи or spreadsheets can be time-consuming and error-prone. This application solves the problem by offering a digital, automated, and user-friendly way to monitor expenses in real time.

3. Application Overview

The Expense Tracker is a user-centric application that allows users to:

Add, edit, and delete expense records
Categorize expenses (e.g., food, travel, bills)
View summaries and reports
Monitor spending trends over time

It provides both daily tracking and overall financial insights

4. Data Handling Approach

Data is stored using local storage / database (e.g., SQLite, Firebase, or backend server).
Each expense record contains:
Amount
Category
Date
Description
CRUD operations (Create, Read, Update, Delete) are implemented.
Data validation ensures accurate input (e.g., no negative values).

5. UI Components

Key UI components include:

Input fields (amount, category, date, description)
Buttons (Add, Update, Delete)
Expense list/table view
Category filter dropdown
Dashboard (charts/summary cards)
Navigation bar/menu

The UI is designed to be clean, responsive, and easy to use.

6. Interactivity Implementation

Dynamic updates when expenses are added or removed
Filtering expenses by category/date
Real-time calculation of total expenses
Interactive charts (pie chart, bar graph)
Responsive design for mobile and desktop

7. Application Logic

Input validation before storing data
Automatic calculation of totals and category-wise expenses
Sorting expenses by date or amount
Generating summaries (daily, weekly, monthly)
Handling edge cases (empty inputs, invalid values)

8. Technology Used

Frontend: HTML, CSS, JavaScript (or React/Angular)
Backend (optional): Node.js / Django / Flask
Database: LocalStorage / MongoDB / Firebase / MySQL
Visualization: Chart.js / D3.js

9. Best Practices Followed

Modular code structure
Separation of concerns (UI, logic, data)
Input validation and error handling
Responsive design principles
Reusable components
Clean and readable code with comments

10. Key Features Implemented

Add/Edit/Delete expenses
Category-based tracking
Expense summaries and analytics
Data persistence
User-friendly interface
Visual reports (charts/graphs)

11. Expected Output / Deliverables

Fully functional expense tracking application
Dashboard with expense summary
Categorized expense reports
Clean and responsive UI
Documentation and source code

12. Learning Outcome

Understanding of CRUD operations
Experience in UI/UX design
Knowledge of data handling and storage
Implementation of real-time interactivity
Exposure to frontend/backend integration

13. Conclusion

The Expense Tracker application provides an efficient and practical solution for managing personal finances. It simplifies expense tracking, improves financial awareness, and helps users make better spending decisions. The project also demonstrates strong fundamentals in application development, including UI design, data management, and interactive features.

CODING

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Expense Tracker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f4f4;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .container {
      background: #fff;
      padding: 20px;
      border-radius: 10px;
      width: 350px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    h2 {
      text-align: center;
    }
    input, button {
      width: 100%;
      padding: 10px;
      margin: 5px 0;
    }
    ul {
      list-style: none;
      padding: 0;
    }
    li {
      display: flex;
      justify-content: space-between;
      background: #eee;
      margin: 5px 0;
      padding: 8px;
      border-radius: 5px;
    }
    .income { color: green; }
    .expense { color: red; }
  </style>
</head>
<body>

<div class="container">
  <h2>Expense Tracker</h2>

  <input type="text" id="desc" placeholder="Description">
  <input type="number" id="amount" placeholder="Amount (+income, -expense)">
  <button onclick="addTransaction()">Add</button>
```

```
<h3>Balance: ₹<span id="balance">0</span></h3>

  <ul id="list"></ul>
</div>

<script>
  let transactions = [];

  function addTransaction() {
    const desc = document.getElementById('desc').value;
    const amount = parseFloat(document.getElementById('amount').value);

    if (!desc || isNaN(amount)) {
      alert('Please enter valid data');
      return;
    }

    const transaction = { id: Date.now(), desc, amount };
    transactions.push(transaction);

    document.getElementById('desc').value = '';
    document.getElementById('amount').value = '';

    render();
  }

  function render() {
    const list = document.getElementById('list');
    const balanceEl = document.getElementById('balance');

    list.innerHTML = '';

    let balance = 0;

    transactions.forEach(t => {
      balance += t.amount;

      const li = document.createElement('li');
      li.innerHTML = `
        ${t.desc}
        <span class="${t.amount >= 0 ? 'income' : 'expense'}">
         ₹${t.amount}
        </span>
      `;

      list.appendChild(li);
    });

    balanceEl.innerText = balance;
  }
</script>

</body>
</html>
```